HackRF 桌面闭环通信实验路线图

适用前提：

- 目标是学通信链路，不是做空口发射
- 默认只做 `线缆 + 衰减器 + 接收机` 的桌面闭环
- 不讨论上天线、不讨论公共空口

这份路线图的核心观点很简单：

- **这些编码不是只能拿来做遥控车**
- 它们是“现代无线链路的地基”
- 用 HackRF 最值得做的是把最基础的几层吃透，而不是一上来追复杂协议

---

1. 这些编码到底能拿来干嘛

`OOK / ASK`

你可以把它理解成最简单的“有/无”或“强/弱”发射。

现实里最像它的东西：

- 便宜遥控器
- 门铃
- 某些早期无线传感器
- 一些极简低速遥测链路

它的价值不是“高级”，而是：

- 门槛最低
- 最容易看到波形和门限判决
- 最容易理解“码元、比特、前导码、判决阈值”这些基础概念

`2-FSK / GFSK`

这比 ASK 更像“正经无线链路”的起点。

现实里和它亲缘很近的东西：

- 各种低速遥测
- 某些老式数传模块
- 很多低功耗无线协议的物理层直觉基础

它会逼你开始理解：

- 频偏
- 带宽
- 符号率
- 接收机鉴频

`BPSK / QPSK`

到这一步，你就已经从“简单无线开关”走向“现代数字调制”了。

现实世界里，大量真正的数字通信系统都和 PSK/QPSK 有直接关系，包括：

- 卫星通信
- 数字微波
- 软件无线电课程里的标准实验
- 很多更复杂调制的基础版本

它的训练价值在于：

- 星座图
- 载波恢复
- 定时恢复
- 相位模糊

这几样一旦理解，后面的很多调制就不再是黑箱。

---

2. 为什么不建议一开始就做“复杂协议”

像下面这些东西，看起来更酷，但不适合做第一步：

- `LoRa`
- `OFDM`

- “仿 Wi-Fi”
- 带完整包结构、同步、CRC、FEC 的复杂链路

原因不是它们不重要，而是它们会把太多问题一次性堆给你：

- 调制本身
- 成帧
- 同步
- 频偏
- 时钟偏差
- 误码
- 信道估计

结果通常是：

- 你看到了一个复杂系统在坏
- 但不知道坏在物理层哪一层

所以最好的路线是：

1. 先把 `ASK` 跑通
2. 再把 `FSK` 跑通
3. 再碰 `BPSK/QPSK`
4. 最后才碰“像系统”的东西

---

3. 你现在缺什么，才能做成“完整通信实验”

如果你手里只有一台 `HackRF One`，有个现实限制：

- **它是半双工**
- **不能同时发和收**

所以要做桌面闭环，最少还缺一个“第二接收端”。

最低成本方案

- 一台 `HackRF One` 做发射
- 一台便宜接收机做接收，比如 `RTL-SDR Blog V4`

这套已经足够做：

- ASK
- FSK
- 很多基础包格式观察

更舒服的方案

- `HackRF One` 做发射
- 另一台更稳一点的接收 SDR 做接收，比如 `Airspy` / `SDRplay` / 第二台 `HackRF`

必备配件

- `30 dB` 到 `60 dB` 衰减器
- 同轴线
- 适配接头

强烈建议

- 准备一个假负载或固定衰减链
- 永远先走线缆，不要先上空口

---

4. 路线图：按三步走

第一步：`OOK / ASK` 闭环

目标

用最简单的方式验证：

- 发射端能发比特流
- 接收端能看到包络变化
- 你能恢复出原始字节

建议实验

- 固定前导码：`10101010`
- 再加一个短 payload，比如 `HELLO` 的字节序列
- 符号率从低开始，例如 `1 kbps` 到 `10 kbps`

你会学到什么

- 什么叫码元
- 什么叫门限判决
- 为什么前导码重要
- 为什么接收机 AGC 会影响解码

成功标准

- 频谱里能看到明显载波/包络变化
- 时域或包络里能看出比特节奏
- 你能稳定恢复出固定 payload

第二步：`2-FSK / GFSK` 闭环

目标

从“幅度表示比特”升级到“频率偏移表示比特”。

建议实验

- 准备两种频偏，例如 `+/- 5 kHz`
- 保持较低符号率，例如 `2.4 kbps`、`4.8 kbps`
- 先做裸 bitstream，不急着上复杂包

你会学到什么

- 什么是频偏
- 为什么符号率和频偏要匹配
- 为什么滤波会影响眼图/码间干扰
- GFSK 为什么比硬切换 FSK 更平滑

成功标准

- 频谱上能清楚看到 FSK 的偏移特征
- 接收端能用简单鉴频恢复出 bitstream
- 在小范围调符号率/频偏时，你能明显看到误码变化

第三步：`BPSK / QPSK` 闭环

目标

进入真正“现代数字调制”的门槛。

建议实验

- 先做 `BPSK`
- 跑通以后再做 `QPSK`
- 符号率不要太高，先追稳定，再追快

你会学到什么

- 星座图是什么意思
- 为什么会有相位翻转/相位模糊
- 为什么载波恢复、定时恢复是核心难点
- 为什么“看得到信号”和“解得出比特”完全是两回事

成功标准

- 星座图稳定成团
- 经过同步恢复后能稳定解码固定 payload
- 你能解释频偏、相位噪声、定时偏差分别会把星座搞成什么样

---

5. 每一步都建议保留的“最小协议”

不要上来就搞复杂协议栈，保留一个简单统一的包格式就够了：

1. 前导码
2. 同步字
3. 长度
4. payload
5. CRC

这 5 样已经足够你理解：

- 接收同步
- 包边界
- 有效载荷
- 误码检测

它比“只发裸字节”更接近真实链路，但又不会复杂到失控。

---

6. 推荐的软件思路

如果你只是做桌面实验，最实用的组合通常是：

- 发射端：`GNU Radio` 或现成基带脚本
- 接收端：`GNU Radio` / `SDRangel` / 自己的解码脚本

`SDRangel` 更适合：

- 看频谱
- 看瀑布
- 看大体效果

`GNU Radio` 更适合：

- 真正搭调制/解调链路
- 做同步、判决、包处理

所以对通信实验来说，`HackRF + GNU Radio` 会比 `HackRF + SDRangel` 更像正路。

---

7. 这东西除了“遥控车”还能训练什么

如果把视野放大一点，这三步其实分别在训练：

- `ASK`：最基础的数字开关键控思维
- `FSK`：最基础的稳健数传思维
- `PSK/QPSK`：现代数字通信思维

它们对应的不是一个具体玩具，而是很多系统里反复出现的共性问题：

- 怎么表示 0 和 1
- 怎么在噪声里恢复 0 和 1
- 怎么找到包开始
- 怎么判断有没有错
- 怎么在带宽、功率、复杂度之间折中

这些东西一旦吃透，你再去看：

- 低速遥测
- 传感器链路
- 数字对讲
- 卫星下行
- 更复杂的 SDR 协议

就会轻松很多。

---

8. 对你现在最务实的建议

如果你真要开始，我建议这样定顺序：

1. 先补一个第二接收机
   `RTL-SDR Blog V4` 就够用
2. 先买衰减器，不要先买功放
   你现在缺的是“安全闭环”，不是更远的发射距离
3. 先做 `OOK/ASK`
   一周内就能跑出第一版闭环
4. 然后做 `2-FSK`
5. 最后再碰 `BPSK/QPSK`

这条线最稳，也最像真的在学通信，而不是在堆功能。

---

9. 一句话总结

如果你把 `HackRF` 的发射能力用在桌面闭环实验上，它不是“拿来做遥控车的小玩具”，而是一块可以把数字通信基础完整走一遍的实验板；真正该追求的不是“发出去”，而是“把链路每一层看明白”。